# EXPERIMENT NO: 6

**Student Name:** Keshav Datta
**UID:** 23BCS10423
**Branch:** BE-CSE
**Section/Group:** KRG-2B

**Semester:** 6th
**Date of Performance:** 25/2/26

**Subject Name:** System Design
**Subject Code:** 23CSH-314

## 1) Aim:

To design a highly scalable video streaming platform similar to YouTube that supports adaptive bitrate streaming (HLS/DASH), dynamic manifest handling, live streaming with sliding window playback, and real-time engagement features (likes and comments) with high availability and low latency.

## 2) Objectives:

- Understand Adaptive Bitrate Streaming using HLS and DASH protocols.
- Implement scalable like and comment systems for static videos.
- Apply event-driven architecture for high-volume engagement processing.
- Ensure fault tolerance, high availability, and eventual consistency.

## 3) Procedure:

1. Studied real-world streaming protocols including HLS and DASH.
2. Analyzed manifest file structure (.m3u8 and .mpd) and adaptive bitrate switching.
3. Designed comment service using NoSQL partitioning for large-scale videos.
4. Evaluated CDN-based global content distribution for low-latency playback.

## 4) Functional Requirements:

- The system should allow users to create an account and log in securely.
- Users should be able to upload videos to the platform. After uploading, the system should process the video and convert it into different qualities so that it can be streamed smoothly.
- Users should be able to watch videos without interruption. The video quality should automatically change depending on the internet speed.
- The platform should allow users to like, dislike, and comment on videos.
- Users should be able to search for videos using keywords.
- The system should provide recommended videos based on user activity and watch history.
- The system should also maintain view counts and interaction data.

## 5) Core Entities of the System

- User - stores user details such as user ID, name, email, password, and creation date.
- Video - stores video information such as video ID, uploader ID, title, description, thumbnail, streaming URL,
- status, and view count.
- Comment - stores comment ID, video ID, user ID, comment text, and timestamp.
- Interaction - stores user ID, video ID, type of interaction (like/dislike), and time.
- WatchHistory - stores user ID, video ID, watched duration, and timestamp

## 6) Core API Endpoints

1. Authentication API

- Register: POST /auth/register
- Login: POST /auth/login

2. Video & Feed API

- Initialize Upload: POST /videos
- Get Video Metadata: GET /videos/{video_id}
- Search videos : GET /search?q=keyword
- Fetch video playlist : GET/stream/{video_id}
- Delete Videos: DELETE /videos/{video_id}

3. Interaction API

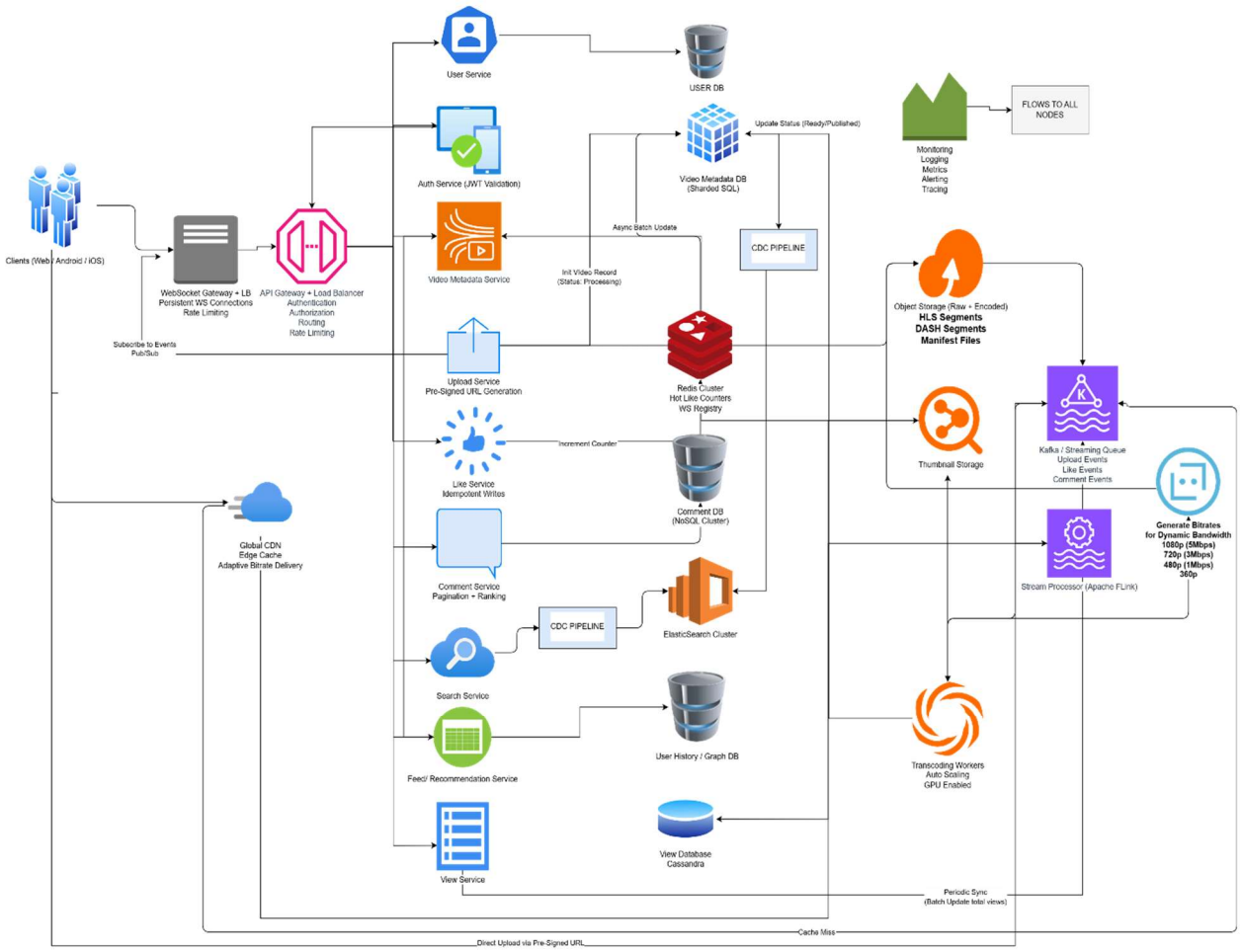
- Like Video: POST /videos/{video_id}/like
- Add Comment: POST /videos/{video_id}/comments
- Get Comments: GET /videos/{video_id}/comments

# 6) Non-Functional Requirements

- High Availability: The system should remain operational even if some components fail.
- Low Latency: Videos should start quickly and buffering should be minimal.
- Scalability: The system should handle millions of users and scale during peak traffic.
- High Throughput: The platform must support large video uploads and high streaming traffic simultaneously.
- Consistency: Video uploads must be strongly consistent, while likes and views can follow eventual
- consistency.
- Security: All communication must be secured using authentication and HTTPS.

.

## 7) High Level Design (HLD)




## 8) Outcome

- Designed a scalable YouTube-like architecture supporting HLS and DASH.
- Implemented dynamic manifest refresh for live streaming.
- Designed sliding window mechanism for low-latency.
- Applied event-driven architecture for engagement processing.